

Private Pre-Shared Key Sync with Azure Active Directory Guide

by Tim Smith, SA – 7/8/2026 – v3.0.1

Overview:

This guide covers how to set up and run the script to sync your Azure (cloud-based) Active directory users with the Private Pre-shared-key (PPSK) within ExtremeCloud IQ (XIQ) public cloud only. PPSK is a solution provided by Extreme Networks to fill the gap between a Wi-Fi SSID solution using a single PSK for all users and deploying a complete 802.1X solution. Extreme Networks PPSK solution allows creating of a dedicated key for each user or device on the same SSID, lowering the number of SSIDs broadcasting in the air and minimizing the airtime consumption due to overhead management frames. This solution also adds the ability to assign VLANs based on user/device groups to avoid the need for separate SSIDs to segregate these groups.

This guide enables you to leverage your existing Azure Active (Entra) Directory security groups to automatically create a Private Pre-shared key for every AD user and remove the PPSK user if a user is disabled or removed from the group in Azure AD.

Each AD user must have a unique email address for this script to work correctly.

Target Audience: Technical

PPSK Use Cases:

- Identity for IoT devices
- BYOD for employees
- Staff device onboarding
- Secure Guest Onboarding (time-based keys with employee sponsorship)
- Hospitality vertical using the hyper-segmentation feature, Private Client Groups (PCGs)
- Third-party via API integration

Table of Contents

Overview	1
Target Audience	1
PPSK Use Cases:	1
Prerequisites:	4
Scripting Environment Preparation:	4
Information:	4
Device Choice:.....	4
Python Installation:.....	4
.....	4
Mac OSX Sonoma	5
Windows 11.....	5
Required Modules:.....	5
Checking for existing Modules	5
Installing required modules	5
.....	5
Script Variables:	6
PCG Support (optional)	6
Generating the XIQ Token	7
Swagger	7
Login	8
Authorize in Swagger	8
Generating Specific Tokens	10
Creating the Azure AD App	11
Granting Permissions to the app	11
Creating Client Secret	12
AD Group Object ID	13
XIQ User Group ID	13
XIQ Network Policy ID	14
Running the Script:	15
Log File	15
Scheduling Script to Run	15
Mac & Linux-based Systems	15
Setting up a Cron Job	15
Cron Job Time Format	17
Cron Job Script and Script Location	17
Cron Job Output and Job Completion	17
Cron Job Command Example	18
Windows based Systems	18
Setting up Windows Task Scheduler	18

Start a Program	18
Editing the Time	19
Troubleshooting:	20
Log File	20
Invalid XIQ token	20
Invalid XIQ token format	20
Expired XIQ token – Code 401 & JWT expired	20
Invalid XIQ Username/password – Code 401	20
Failed Authentication for Azure AD API	20
Invalid or Wrong Group ID entered	20
XIQ User Failed to Create – Code 400	21
XIQ Timeout Error	21
Email is not set for AD User	21
Azure_AD_Test.py.....	21

Prerequisites:

- ExtremeCloud IQ Public Cloud, Private Cloud (IQVA on-prem is not supported.)
 - o The key directory can be stored in the cloud (unlimited keys) or local on all access points (10,000 maximum key limit)
- Knowledge of XIQ by adding access points, creating network policies, and SSIDs
- XIQ PPSK SSID and associated User Groups configured
- RadSec Proxy requires TCP Port 2083 to be open on your internet firewall
- One or more XIQ native access points
- Not supported on wired systems, A3 NAC, or campus-based Wi-Fi systems (WiNG or IdentiFi)
- Download the following files
 - XIQ-Azure-AD-PPSK-Sync.py
 - Version 3.1.0 is the current version. See lines 10-15 in the script
 - Azure_AD_Test.py (optional – see [troubleshooting](#) section)
 - Version 1.1.0 is the current version. Lines 6-11
 - requirements.txt (optional – see [modules](#) section)

```
#####  
# written by:  Tim Smith  
# e-mail:     tsmith@extremenetworks.com  
# date:       12 June 2026  
# version:    3.0.1  
#####
```

Scripting Environment Preparation:

Information:

The XIQ-Azure-AD-PPSK-Sync.py script requires, at minimum, Python 3.6 and tested up to Python 3.13. This script can be executed manually but ideally would be set up as a cronjob to be run every 8, 12, or 24 hours. This script can be executed from any device with Python and the needed modules installed. This device must reach the Azure Cloud and reach out to ExtremeCloud IQ.

The script, when run, will create a XIQ-Azure-AD-PPSK-Sync.log file. This log file will show information about PPSK users created and deleted. It will also show how many users were parsed from XIQ and Active Directory when run. Any API errors experienced will also show up in the log file.

Device Choice:

This script can be executed from any device running Python 3.6 or higher. The device could be a server running Redhat, a pc/laptop running Windows 11 or Mac OSX, or even a Raspberry Pi-type device. The device will need to be on the network and be able to reach the Azure Cloud as well as reach ExtremeCloud IQ. This can be done through a proxy. Proxy config is beyond the scope of this guide.

Python Installation:

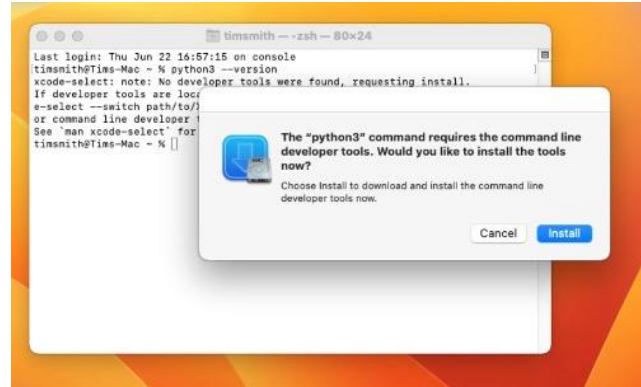
Depending on the device used, you may need to install Python or a different version of Python. The easiest way to check the version of Python is to open the terminal (Power Shell on Windows) and type this command.

```
python3 --version
```

Below are some examples of installing Python3 for Windows and Mac OSX. Linux systems that were tested all had Python3.6 or higher installed by default.

Mac OSX Tahoe

- Open the terminal and enter 'python3 --version'
 - This triggers the installation of Developer Tools
- Click Install
- Click Agree
- The Developer tools that installed python3 will also install pip3 in Sonoma
- Mac terminal will be used to install python modules



Windows 11

- Search Microsoft Store for Python 3.12 and click install
- Log in with Microsoft credentials
- The Windows store installs pip3 with python3. Pip3 will be used to install the needed modules.
- Windows power shell or command prompt can be used to install Python modules



Required Modules:

The **requests** module is the only module required for the XIQ-Azure-AD-PPSK-Sync.py script. For Linux and Mac

Checking for existing Modules

You can check if the required module is installed using the terminal (PowerShell for Windows). Run the following command.

```
python3 -c "import requests"
```

The module is not installed if a '*ModuleNotFoundError: No module named '<module name>'*' error is returned.

Installing required modules

The required module can be installed using pip3 using the downloaded requirements.txt file with the following command.

```
pip3 install -r requirements.txt
```

Or the module can be installed individually using.

```
pip3 install requests
```

Script Variables:

The Global Variable section of the script (file name: XIQ-Azure-AD-PPSK-Sync.py) must be updated with the correct values. We will briefly cover each of these and, for some, will go into more detail below.

Lines 19 – 21 are info for Azure Active Directory

1. **tenant_id** – Azure Directory (tenant) ID created when creating the Azure App. See [creating the Azure app](#).
2. **client_id** – Azure Application (client) ID created when creating the Azure App. See [creating the Azure app](#).
3. **client_secret** - Azure Client Secret created when creating the Azure App. See [creating the Azure app](#).

Line 24 is the number of PPSK and PCG users to be returned per API call.

Lines 26 – 30 are for XIQ authentication. 2 methods could be used, but the token method is recommended.

1. XIQ username and password –
 - **Lines 26 and 27** - uncomment by deleting the # at the beginning of the line. Then fill in the username and password.
 - **Line 30** - comment out the line by adding # to the beginning of the line.
2. Token method – a token can be generated to only allow access to view/create/delete PPSK users. This is the preferred method. Details on generating this token are below in the [Generating the XIQ Token, specifically](#) the [Generating Specific Tokens](#) sub-section.

```
# Global Variables - ADD CORRECT VALUES
tenant_id = 'Azure Directory (tenant) ID'
client_id = 'Azure Application (client) ID'
client_secret = 'Azure Client Secret'

#XIQ MaxPageSize (max is 100)
pageSize = 100

#XIQ_username = "enter your ExtremeCloudIQ Username"
#XIQ_password = "enter your ExtremeCloudIQ password"
####OR####
## TOKEN permission needs - enduser, pcg:key, lro
XIQ_token = "****"

group_roles = [
    # AD GROUP Object ID, XIQ group ID
    ("AD Group Object ID", "XIQ User Group ID"),
    ("AD Group Object ID", "XIQ User Group ID")
]
```

```
# Global Variables - ADD CORRECT VALUES
tenant_id = '287542ec-a80c-4b35-ae42-2e7134c1062c'
client_id = 'c2d57b19-cc0f-4589-aece-b38d3656997e'
client_secret = 'Fio8Q~.A6uAihMBmd.T9f9VK3zscgxA9SgBBhBE-'

#XIQ MaxPageSize (max is 100)
pageSize = 100

#XIQ_username = "enter your ExtremeCloudIQ Username"
#XIQ_password = "enter your ExtremeCloudIQ password"
####OR####
## TOKEN permission needs - enduser, pcg:key
XIQ_token = "eyJraWQwI0IiOiJhZlZDM3NTVjY2U0YWZzODg5MTY5N2YyJFIMGUzZCIsIn"

group_roles = [
    # AD_GROUP Name, XIQ group ID
    ("e7274e17-d57f-4cb3-b3e5-fe7231f952e5", "769490635890716")
]
```

Lines 32 – 36 are where we will define which AD groups to sync with which XIQ User Groups. Note that the brackets around the groups create a list. Each set of groups should be in a set of parentheses; a comma should separate each element, and a coma should separate each set of groups. If only one set of groups is required, **line 35** can be deleted, and the comma at the end of **line 34** removed.

- We will cover how to get the needed AD Group Object in the [AD Group Object ID](#) section.
- We will cover how to get the XIQ User Group ID in the [XIQ User Group ID](#) section.
- **NOTE:** The order is very important here. If the same AD user is in multiple groups, the user will be put in the first XIQ User Group in the list. XIQ users can only be in one PPSK User group.

PCG Support (optional)

Line 38 To enable PCG Support, change the **PCG Enable** Variable from **False** to **True**.

Lines 40-46 If PCG is Enabled, **PCG_Mapping** should be updated with the correct information. If PCG is not Enabled, **PCG_Mapping** will not be used and does not need to be updated.

1. **Line 41** – This should be replaced with the XIQ User Group ID number that correlates with the PCG. See [XIQ User Group ID](#) section.

2. **Line 42** – This is the name of the User Group associated with the ID on line 42
 - This is needed to add and remove Users from the PCG.
3. **Line 43** – This is the Network Policy ID associated with the PCG
 - We will cover how to get the Network Policy ID in the [Network Policy ID](#) section
4. **Line 44** – This is the Network Policy Name associated with the PCG

```
PCG_Enable = False

PCG_Mapping = {
  "XIQ User Group ID" : {
    "UserGroupName": "XIQ User Group Name",
    "policy_id": "Network Policy ID associated with PCG",
    "policy_name": "Network Policy name associated with PCG"
  }
}

extended_username_format = False # Set to True if you want to use the e
# If True, name would be <Active Directory username>_<email>
# If False, the name would be just the <Active Directory username>
```

```
PCG_Enable = True

PCG_Mapping = {
  "769490635890716" : {
    "UserGroupName": "PCG_Users",
    "policy_id": "769490635980793",
    "policy_name": "PCG_Test"
  }
}

extended_username_format = False # Set to Tr
```

Line 48 – When this is True, the PPKS user name will combine the AD Azure display name and email address. – <displayName_email> This allows the script to work when multiple AD users have the same display name with different email addresses.

Generating the XIQ Token

You can view our developer portal site at <https://developer.extremecloudiq.com/> There is a link to our swagger page here and other developer tools. There is also a Communities section to reach out to with any questions.

Swagger

We will use the swagger interface to generate the token <https://api.extremecloudiq.com/>.

On the swagger page, clicking on any API will expand information about the API and allow you to try it. Clicking the “Try it out” button, filling out any needed information, and then clicking the execute button will allow you to try that specific API call.

The 2nd generation APIs are based on access tokens generated by a XIQ account. External XIQ accounts need to be granted API to leverage the new APIs. See [A Guide to Getting Started with v2 APIs in XIQ](#) for more details. Currently, these tokens can only be generated through the `/login` POST API request and cannot be generated through the XIQ GUI.

```
{
  "username": "xiq@example.com",
  "password": "changeme"
}
```

[illegible]

extremecloudiq.com - ExtremeCloud IQ REST API Server

Authorize

Authentication

Authenticate user to access protected resources

login

Basic authentication

Authentication

API token and permissions

auth/apitoken

Generate new API token

auth/apitoken/info

Get current API token

auth/permissions

Get permissions for user

auth/permissions/:check

Check permissions for user

ExtremeCloud IQ Account

Get ExtremeCloud IQ account information

account/info

Get ExtremeCloud IQ account information

account/viq

Get VIQ Info

Users, local and external user management

Available authorizations

BearerAuth (http, Bearer)

JSON Web Token (JWT) based authentication

Value:

Authorize

Close

Generating Specific Tokens

POST /auth/apitoken Generate new API token 

The `/auth/apitoken` POST request allows you to specify an expiration time and set permissions for a token. This is a great way to create a token for a specific application or script, only allowing the token to perform the needed tasks.

The expiration time uses Epoch time, which is the number of seconds since midnight on Jan 1, 1970 (UTC). <https://www.epochconverter.com/> is a webpage that can convert a readable time to an epoch time or epoch time to a more readable time. Set a time for 1 year out and get the epoch time.

For this script, we will want to have the following permissions - enduser, pcg:key, lro

This will give us access to view, create, and delete ppsk users and view, create, and delete pcg-key-based users if necessary.

Adding the desired expiration time and a list of permissions, this API will return a token only usable by the specified APIs.

Request Body

```
{
  "description": "Token for XIQ-AD-PPSK-Sync.py script",
  "expire_time": 1784989236,
  "permissions": [
    "enduser",
    "pcg:key",
    "lro"
  ]
}
```

Response Body

```
{
  "access_token":
    "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ0aW1qc21pdGgyNEBwcm90b25tYWwSLmNvbSIsInNjb3BlcyI6WyJhY2NvdW50OnliXSwidXNlcklkIjoyMTc5MjMyMSwicz9sZSI6IkFkbWluaXN0cmF0b3IiLCJjdXN0b21icklkIjoyMTc5MTk3MSwiY3VzdG9tZXJNb2RlIjowLCJoaXFFbmFibGVkIjpmYWxzZWwib3duZXJJZCI6MTc5MTYxLCJvcmdJZCI6MCwiZGF0YUNlbnRlcil6IkIBX0dDUClzImV4dHJlbnVjbG91ZGlxLmNvbSIsImhhdCI6MTYyODE4MzA4OSwiZXhwIjoxNjI4MTg2NDI4fQ.CtBGq4YVGB9FzCodr6Oi5IG8yy1-4B-77AWI5rVG3S0",
  "create_time": "2021-11-29T15:47:57.000+0000",
  "expire_time": "2021-11-29T16:10:08.000+0000",
  "creator_id": 21792321,
  "customer_id": 21791971,
  "description": "Token for XIQ-AD-PPSK-Sync.py script",
  "permissions": [
    "enduser",
    "pcg:key",
    "lro"
  ]
}
```

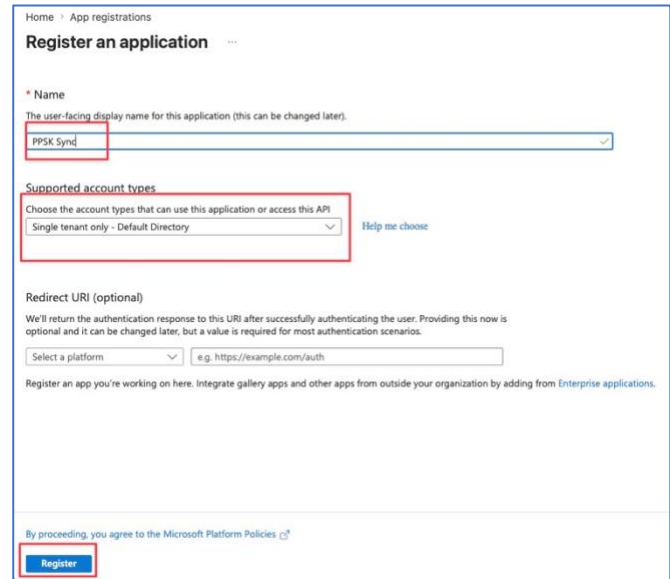
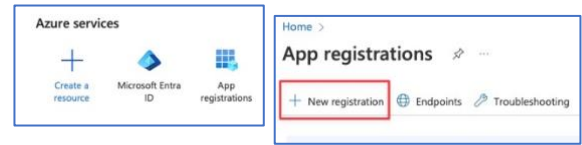
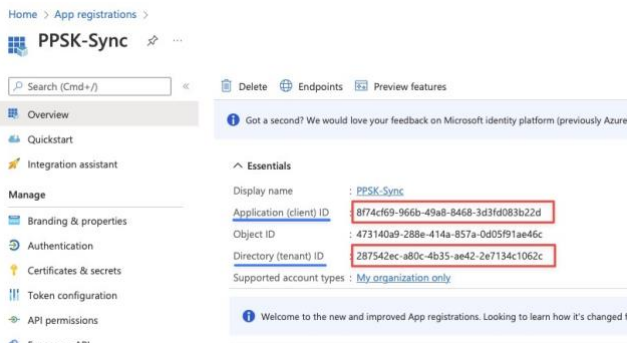
Copy the newly created `access_token` and add it to the `XIQ_token` variable in the script.

Creating the Entra AD App

Select App registration under Azure services in the Azure portal to set up the Entra AD App. This page will list any apps that you have. On the top, select the option to add a **‘+ New registration’**.

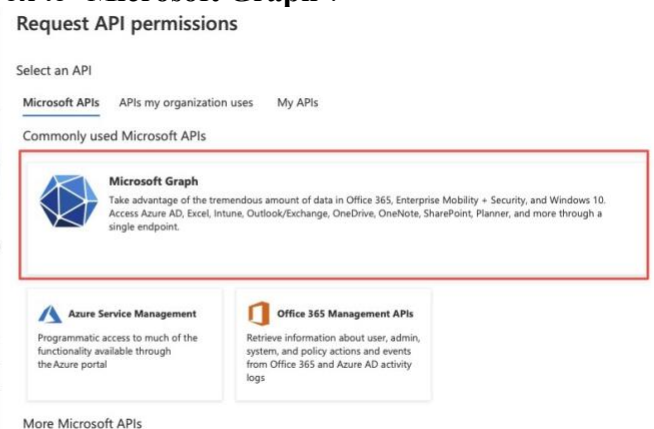
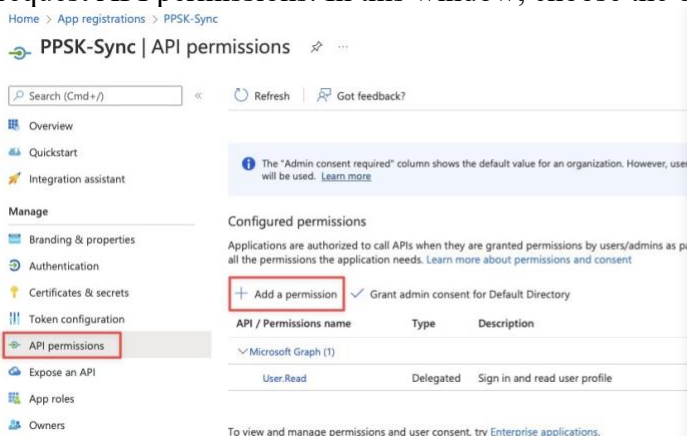
This will open a page where you can give your application a name, select the account type, and register the app.

Once the app is registered, a screen will open with information about the app. Here you can get the **tenant_id** and **client_id**. Copy these items into the script.



Granting Permissions to the app

You will need to add app permissions to collect group and user information. To do this, select **‘API Permissions’** on the left and then select **‘+ Add a permission’**. This will open a window on the right to request API permissions. In this window, choose the box to **‘Microsoft Graph’**.



In the next window, select **'Application permissions'**. This will allow the application to run without redirecting for user authentication.

You can search for 'group', select **'Group.Read.All'**, and click **'Add permission'**. This will grant access to the Azure API to read group information. You will also need to add permission for **'User.Read.All'** to grant access to reading user information through the API.

Follow the same process of selecting **'+ Add a permission'**, selecting the box for **'Microsoft Graph'**, selecting **'Application permissions'**, searching for 'user', selecting **'User.Read.All'**, and clicking **'Add permission'**.

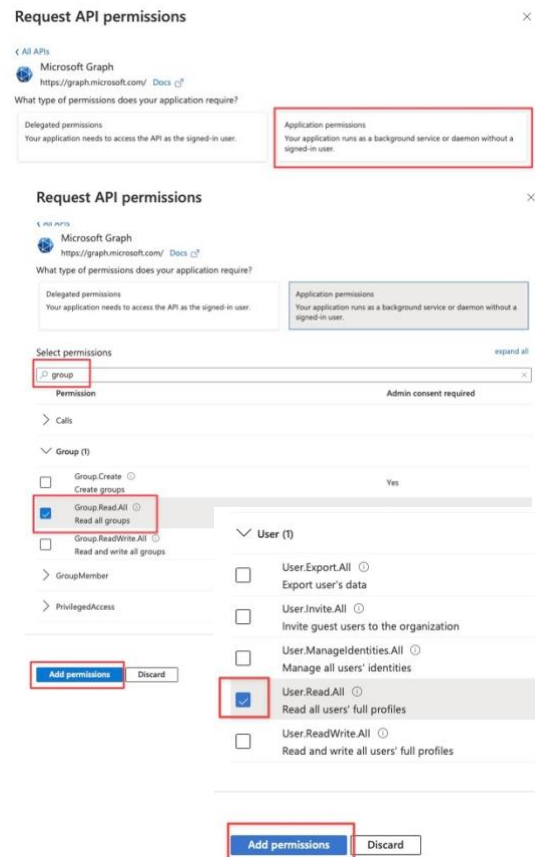
Once the permissions are added, you must grant admin consent for Default Directory. You can do this by selecting the checkmark next to the **'Add a permission'** button.

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission ☒ Grant admin consent for Default Directory

API / Permissions name	Type	Description	Admin consent req.	Status
Microsoft Graph (1)				
Group.Read.All	Application	Read all groups	Yes	☒ Not granted for Default
User.Read	Delegated	Sign in and read user profile	No	☒ Not granted for Default
User.Read.All	Application	Read all users' full profiles	Yes	☒ Not granted for Default



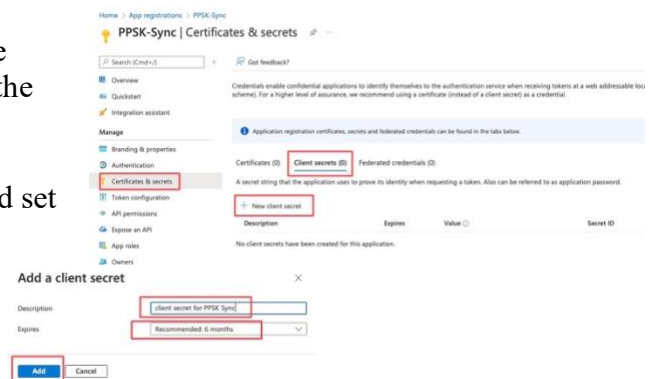
Creating Client Secret

After the permissions are created and granted consent, you can create the client secret. **NOTE:** A new secret must be created if the client secret is created before the above process is completed.

Select the **'Certificates & secrets'** on the left to create the Client Secret. Select the **'Client secrets'** tab, and select the **'+ New client secret'** button.

A screen will open allowing you to add a Description and set a time for the 'client secret' to expire.

Once the client secret is created, you can copy the The value that is created and add it to the script.



Certificates (0) **Client secrets (1)** Federated credentials (0)

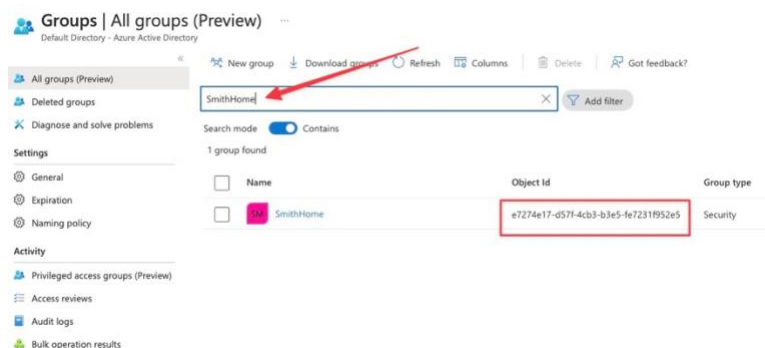
A secret string that the application uses to prove its identity when requesting a token. Also can be referred to as application password.

+ New client secret

Description	Expires	Value	Secret ID
client secret for PPSK Sync	12/22/2022	ZwBQ-Ld-ksof77_TwbRvqg75OmOOh	af156995-2378-4aab-ad62-55269666961

AD Group Object ID

Each group in Azure AD will have a unique Object ID assigned to it. An easy way to get this ID is to log into your Azure portal, enter Azure Active Directory, and select Groups. You can search for the group by name in the group section. In the list of groups, each group will have a unique Object ID displayed in the Object ID column.



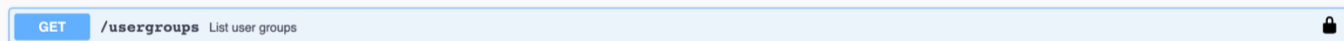
Once the Object ID is obtained, add it to the group_roles object in the script.

NOTE: The script will collect all users below the group entered. If nested groups are used, note that all users under the nested group will also be part of the PPSK user group.

```
group_roles = [
    # AD GROUP Object ID, XIQ group ID
    ("e7274e17-d57f-4cb3-b3e5-fe7231f952e5", "XIQ User Group ID"),
    ("AD Group Object ID", "XIQ User Group ID")
]
```

XIQ User Group ID

Each XIQ User Group will be assigned a unique ID when created. This is something that the backend systems use and is not seen in the GUI. The easiest way to get the ID is from the Swagger page. Return to the swagger page, scroll down to the Configuration – User Management section, and find the /usergroups GET request.



Click the “Try it out” button, then the “Execute” button. When you find the Name of the XIQ User Group you want to use, it will be inside a pair of {curly brackets}. Inside the same pair of curly brackets will be an element called id, and this is the ID that is needed.

Response Body

```
{
  "page": 1,
  "count": 10,
  "data": [
    {
      "id": 769490635824436,
      "name": "Home_Hive",
      "description": "",
      "predefined": false,
      "create_time": "2021-10-11T18:24:33.000+0000",
      "update_time": "2021-10-11T18:24:33.000+0000",
      ...
    }
  ]
}
```

Once that is obtained, add it to the group_roles object in the script.

```
group_roles = [
    # AD GROUP Object ID, XIQ group ID
```

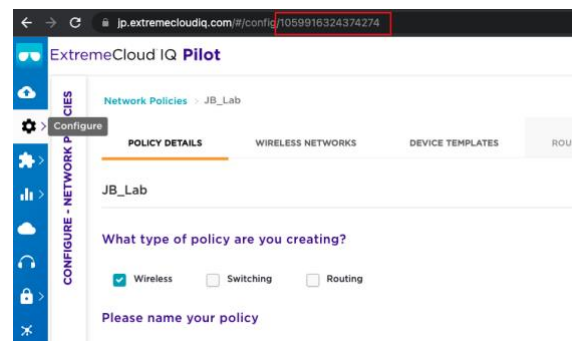
```
(
  "e7274e17-d57f-4cb3-b3e5-fe7231f952e5",
  "769490635824436",
  ("AD Group Object ID", "XIQ User Group ID")
)
```

If needed, continue to gather other AD Group Distinguished Names and XIQ User Group IDs. Enter them in the same format, with each set enclosed in parentheses. All but the last one should be followed by a coma.

```
group_roles = [
  # AD GROUP Object ID, XIQ group ID
  ("d1ba206a-0213-4ae1-95de-0778b366b778", "769490635824438"),
  ("e7274e17-d57f-4cb3-b3e5-fe7231f952e5", "769490635824436"),
  ("27a8d84c-f4c4-41e2-a84e-9abdac7c0c13", "769490635824395")
]
```

XIQ Network Policy ID

Each XIQ Network Policy will be assigned a unique ID when created. The easiest way to get the ID is to select the Network Policy in the XIQ GUI. When you select the Network Policy, the ID is the long number in the URL, and the Policy Name is directly under the Policy Details.



You can also get the Network Policy ID from Swagger for all Network Policies with PCG configured.

Return to the swagger page, scroll down to the Configuration – User Management section, and find the `/pcg/key-based` GET request. If the Network Policy is configured to use PCG, it will be listed in the response. The policy id and policy name are included in this response.

GET
/pcgs/key-based
List Key-based Private Client Groups

Response Body

```
{
  "id": 1059916324374423,
  "create_time": "2021-12-13T15:37:55.000+0000",
  "update_time": "2022-01-06T20:29:15.000+0000",
  "org_id": 0,
  "policy_id": 1059916324374274,
  "policy_name": "JB_Lab",
  "ssid_name": "PCG_Test",
  "enabled": true,
  "users": [
  ]
}
```

Running the Script:

To run the script, open the terminal (PowerShell for Windows) to the location of the script and run:

```
python3 XIQ-Azure-AD-PPSK-
```

You can also make the script executable by running

```
chmod +x XIQ-Azure-AD-PPSK-Sync.py
```

Then you can run the script by typing

```
./XIQ-Azure-AD-PPSK-
```

The script will print to the screen how many PPSK users and AD users were parsed. If there are any users in the list of AD users and not in the list of PPSK users, an API call will be made to create the PPSK user. The script will print on the screen for each user it successfully creates.

```
successfully created PPSK user Tim Smith
```

If there are any users in the list of PPSK users that are not in the list of AD users or in the list of disabled AD users, an API call will be made to delete the PPSK user. The script will print on the screen for each user it successfully deletes. – This will be the email address of the user.

```
User user0200@example.com - 769490635839948 was successfully deleted.
```

The AD user must have an email address assigned, or the PPSK user will not be created. A message will print on the screen.

```
User Ruby Smith doesn't have an email set and will not be created in xiq
```

Log File

Upon running the script, a log file will be created named XIQ-Azure-AD-PPSK-Sync.log. Additional runs of the script will append to this log file. This file will contain the same information that prints to the screen and any error received when making the API calls. This is an excellent place to look if issues are seen.

```
40 2022-06-22 14:07:50: root - INFO - Successfully parsed 4 Azure AD users
41 2022-06-22 14:07:50: root - INFO - Successfully parsed 4 XIQ users
42 2022-06-22 14:07:50: root - INFO - Successfully parsed 4 Azure AD users
43 2022-06-22 14:07:50: root - INFO - Successfully parsed 4 XIQ users
44 2022-06-22 14:07:50: root - INFO - Successfully parsed 3 Azure AD users
45 2022-06-22 14:07:51: root - INFO - User tim.smith@azure-key.com - 43379107700612 was successfully deleted.
46 2022-06-22 14:07:51: root - INFO - Successfully parsed 3 XIQ users
47 2022-06-22 14:07:51: root - INFO - Successfully parsed 5 Azure AD users
48 2022-06-22 14:07:51: root - INFO - successfully created PPSK user Adele Smith
49 2022-06-22 14:07:51: root - INFO - successfully created PPSK user Ruby Smith
50 2022-06-22 14:07:51: root - INFO - Successfully parsed 4 XIQ users
51 2022-06-22 14:07:51: root - INFO - Successfully parsed 5 Azure AD users
52 2022-06-22 14:07:51: root - INFO - successfully created PPSK user Ruby Smith
53 2022-06-22 14:08:01: root - INFO - Successfully parsed 4 XIQ users
54 2022-06-22 14:08:01: root - INFO - Successfully parsed 5 Azure AD users
55 2022-06-22 14:08:01: root - WARNING - User Ruby Smith doesn't have an email set and will not be created in xiq
56 2022-06-22 14:13:50: root - INFO - Successfully parsed 4 XIQ users
57 2022-06-22 14:13:50: root - ERROR - Error getting access token - HTTP Status Code: 401
58 2022-06-22 14:13:50: root - ERROR - Failed to Authenticate with Azure AD - Error getting access token - HTTP Status Code: 401
59 2022-06-22 14:15:51: root - INFO - Successfully parsed 4 XIQ users
60 2022-06-22 14:15:51: root - ERROR - Error retrieving Azure AD users - HTTP Status Code: 400
61 2022-06-22 14:15:51: root - WARNING - {"error":{"code":"Request_BadRequest","message":"Invalid object identifier 'e7274e17-d571-4c93-b3e5-fc7231f952"}}
```

Scheduling Script to Run

Mac & Linux-based Systems

A Cron job can be set up to run the script at a specified interval automatically. Ideally, this should be set for every 8, 12, or 24 hours, ensuring the AD and PPSK user groups stay in sync. The script can also be run manually between those times if a user needs to be added or removed immediately.

Setting up a Cron Job

Open and edit the crontab and configure the job with the arrangement for the command you want to run. From the terminal window, enter the following command.

```
crontab -e
```

There are three parts to a cron job configuration.

Cron Job Time Format

Part 1 of the cron job

The first five characters, a b c d e, represent the job's time, date, and repetition.

- a – Minute (0-59)
- b – Hour (0-23)
- c – Day (0-31)
- d – Month (0-12) – 0=None and 12 = December
- e – Day of the Week (0-7) – 0=Sunday and 7=Sunday

- **An asterisk (*)** stands for all values. Use this operator to keep tasks running during all months, or all days of the week.
- **A forward-slash (/)** is used to divide a value into steps. (* / 2 would be every other value, * / 3 would be every third, * / 10 would be every tenth, etc.)

The time format would look like this to set the Cron job to run every 8 hours.

```
0 */8 ***
```

The time format would look like this to set the Cron job to run every night at midnight.

```
00 ***
```

Cron Job Script and Script Location

Part 2 of the cron job

The next part is where you enter the script you want to run and its location.

```
python3 /home/admin/documents/scripts/XIQ-Azure-AD-PPSK-Sync.py
```

You can make the script executable, so you don't have to type python3 before entering the script. Instead, you will enter a period before the location.

```
chmod +x /home/admin/documents/scripts/XIQ-Azure-AD-PPSK-Sync.py
```

```
./home/admin/documents/scripts/XIQ-Azure-AD-PPSK-Sync.py
```

Cron Job Output and Job Completion

Optional Part 3 of the cron job

The last part is an optional part that specifies where the output and completion of the script should go. If not set, the cron will send an email to the owner of the crontab file.

To avoid filling up the inbox on the server, it is recommended to have something set for the output. This can be set to append to a file.

```
>> /home/admin/documents/scripts/XIQ-Azure-AD-PPSK-Sync-
```

Or can it be set to turn off the email output

```
> /dev/null 2>&1
```

Cron Job Command Example

The command should be entered in a single line and saved in the crontab file.

Every 8 hours turning off the output

```
0 */8 * * * python3 /home/admin/documents/scripts/XIQ-Azure-AD-PPSK-Sync.py > /dev/null 2>&1
```

Every 12 hours with saved output

```
0 */12 * * * ./home/admin/documents/scripts/XIQ-Azure-AD-PPSK-Sync.py >> /home/admin/documents/scripts/XIQ-Azure-AD-PPSK-Sync-Output.txt
```

Windows based Systems

A Windows task schedule can be set up to run the script at a specified interval automatically. Ideally, this could be set for every 8, 12, or 24 hours, ensuring the AD and PPSK user groups would stay in sync. The script can also be executed manually between those times if a user needs to be added or removed immediately.

Setting up Windows Task Scheduler

Open Control Panel > System and Security > Administrative Tools > Task Scheduler

Selected 'Create basic task...'

Give your task a name like 'AD-PPSK-Sync' and click 'Next'

Leave the Trigger set to daily and click 'Next' – We will return and adjust this.

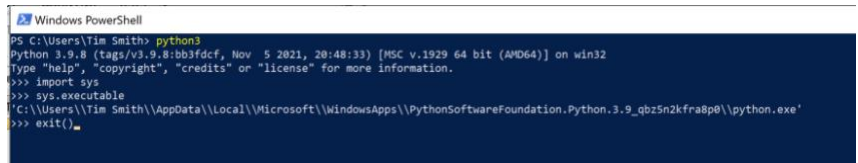
Click 'Next' leaving recur every one day

Select 'Start a program and click 'Next'

Start a Program

For the Program/script: section, enter the path of your python.exe file.

The location of your python.exe file depends on how it was installed. An easy way to find this location is to open Windows PowerShell and enter python3. This will open the Python interpreter. In the interpreter, enter then.



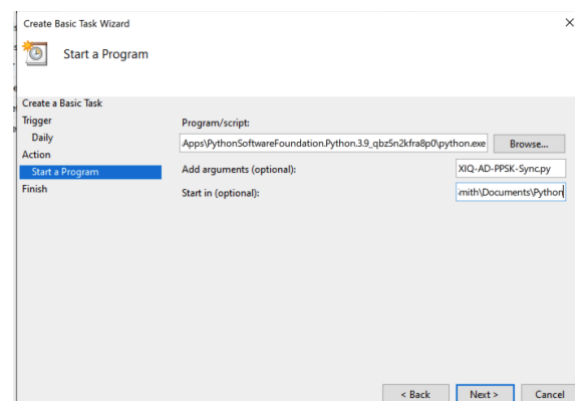
```
PS C:\Users\Tim Smith> python3
Python 3.9.8 (tags/v3.9.8:bb3fdec, Nov 5 2021, 20:48:33) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import sys
>>> sys.executable
'C:\Users\Tim Smith\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.9_qbz5n2kfra8p0\python.exe'
>>> exit()
```

This will output the location of the python.exe file. Enter to exit the interpreter.

Enter the full path of the python.exe file in the Program/Script: field.

Enter the script's name in the Add arguments (optional) field.

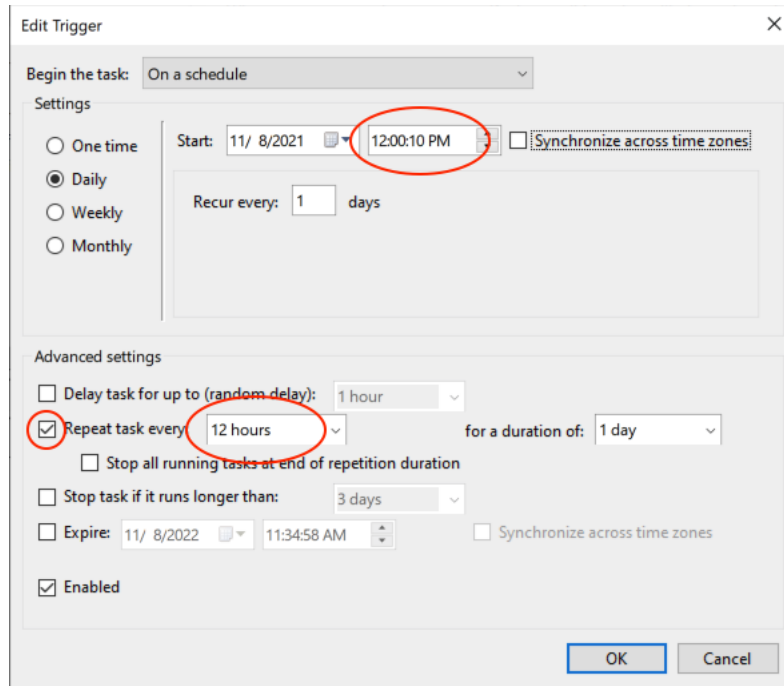
Enter the script's location in the Start in the (optional) field.



Editing the Time

Once the task is saved, open the Task Scheduler Library folder, and find the newly created AD-PPSK-Sync task. Click on it to open, select the Trigger tab, and edit the Daily trigger. Here you can set what time you want it to run.

If you want the script to run every 8 or 12 hours, check the box next to 'Repeat task every:' and enter '8 hours' or '12 hours'



Edit Trigger

Begin the task: On a schedule

Settings

☐ One time

☒ Daily

☐ Weekly

☐ Monthly

Start: 11/ 8/2021 12:00:10 PM ☐ Synchronize across time zones

Recur every: 1 days

Advanced settings

☐ Delay task for up to (random delay): 1 hour

☒ Repeat task every 12 hours for a duration of: 1 day

☐ Stop all running tasks at end of repetition duration

☐ Stop task if it runs longer than: 3 days

☐ Expire: 11/ 8/2022 11:34:58 AM ☐ Synchronize across time zones

☒ Enabled

OK Cancel

Troubleshooting:

Log File

The XIQ-Azure-AD-PPSK-sync.log file is a good place to look for potential issues. This log file will update whenever the script is run, manually or on a schedule.

Invalid XIQ token

```
2021-11-05 16:58:27: root - ERROR - Error retrieving PPSK users from XIQ - HTTP Status Code: 401
2021-11-05 16:58:27: root - WARNING - {"error_code": "AuthInvalidToken", "error_id": "cda656a5157d4c87a5143252aad71bff", "error_message": "Unable to read JSON value:
?[\x19???x14?M??]}
```

Check token using [Swagger](#) - remember that if you generate a specific token, it may only have access to the user's APIs.

Invalid XIQ token format

```
2021-11-08 13:56:36: root - ERROR - Error retrieving PPSK users from XIQ - HTTP Status Code: 401
2021-11-08 13:56:36: root - WARNING - {"error_code": "AuthInvalidToken", "error_id": "555d1ce9f67b40ef83caf4a89ca92b04", "error_message": "JWT strings must contain exactly 2 period
characters. Found: 0"}
```

This may mean you tried to use XIQ Username and Password but have yet to comment out line 28. Add a # in front of **line 28**. Or the token wasn't entered correctly

Expired XIQ token – Code 401 & JWT expired

```
2021-11-08 14:14:16: root - ERROR - Error retrieving PPSK users from XIQ - HTTP Status Code: 401
2021-11-08 14:14:16: root - WARNING - {"error_code": "AuthTokenExpired", "error_id": "78d8b818a03940dd8d4accfc1b3ffb7e", "error_message": "JWT expired at 2021-11-08T19:14:11Z."}
```

The XIQ token has expired. Generate a new token using [Swagger](#).

Invalid XIQ Username/password – Code 401

```
2021-11-08 13:55:15: root - ERROR - Error getting access token - HTTP Status Code: 401
2021-11-08 13:55:15: root - WARNING - <Response [401]>
```

Check username and password for XIQ. – I recommend using XIQ Token

Failed Authentication for Azure AD API

```
2022-06-22 14:13:58: root - ERROR - Failed to Authenticate with Azure AD - Error getting access token - HTTP Status Code: 401
```

Check the tenant id, client id, and client secret.

Invalid or Wrong Group ID entered

```
2022-06-22 14:16:53: root - ERROR - Error retrieving Azure AD users - HTTP Status Code: 400
2022-06-22 14:16:53: root - WARNING - {"error":{"code":"Request_BadRequest","message":"Invalid object identifier 'e7274e17-d57f-4cb3-b3e5-fe7231f952e'.", "innerError":{"date":"2022-06-22T18:16:53","request-id":"353bdc25-3d80-4e6a-9f6f-c2c205707f0a","client-request-id":"353bdc25-3d80-4e6a-9f6f-c2c205707f0a"}}}
```

Check the group ID for the Azure AD group

XIQ User Failed to Create – Code 400

```
2021-11-08 14:19:35: root - INFO - Successfully parsed 0 XIQ users
2021-11-08 14:19:36: root - INFO - Successfully parsed 4 LDAP users
2021-11-08 14:19:36: root - ERROR - Error adding PPSK user Tim Smith - HTTP Status Code: 400
2021-11-08 14:19:36: root - WARNING - {'error_code': 'UNKNOWN', 'error_id': None, 'error_message': 'UNKNOWN'}
2021-11-08 14:19:36: root - ERROR - failed to create Bauer Smith: Error adding PPSK user Tim Smith - HTTP Status Code: 400
```

There are a couple of possibilities when you see this error. As you can see above, 0 XIQ users were parsed, and the user failed to create. In this instance, the XIQ User Group ID needed to be corrected. If the XIQ users parsed was 0 and you have configured users in the user group, check the user group ID in the group_roles list on **Lines 30-34**

The other thing that could cause this Error/Warning is if the username already exists in XIQ PPSK users within a different user group.

XIQ Timeout Error

```
2021-10-08 12:39:29: root - ERROR - Error deleting PPSK user 769490635824383 - HTTP Status Code: 504
2021-10-08 12:39:29: root - WARNING - <Response [504]>
2021-10-08 12:39:29: root - ERROR - Failed to delete user [REDACTED] with error Error deleting PPSK user 769490635824383 - HTTP Status Code: 504
2021-10-08 12:43:05: root - ERROR - Failed retrieving PPSK users from XIQ - HTTP Status Code: 504
2021-10-08 12:43:05: root - WARNING - <Response [504]>
```

An HTTP Status Code: **504** is a timeout from XIQ. If XIQ cannot respond to the API call within 60 secs, it will send these **504** errors. The script will need to be rerun with no changes.

Email is not set for AD User

```
2021-11-08 14:42:39: root - WARNING - User Tim Smith doesn't have an email set and will not be created in xiq
```

Check the AD User and see if the email is set. Email is required to create a PPSK.

Azure_AD_Test.py

This script was written to help troubleshoot issues with collecting data from AD. The variables are the same in this script, except that the AD Group ID should be added to the **ad_group_id** variable instead of in the group_roles list.

This script will test the Azure AD app API. If there are any errors, they will print to the screen. Otherwise, the collected data from Azure AD will print on the screen. If empty [] brackets print, check the [Group ID](#) and validate that it is correct.

Using this output, you can check whether the user is disabled in Azure AD. You can validate that there is an email set for the user. And overall, check that information is being returned.

```
completed page of AD Users. Total Users collected is 5
Tripp Smith {'accountEnabled': True, 'email': 'user0001@example.com', 'username': 'Tripp@example.onmicrosoft.com'}
Miles Smith {'accountEnabled': True, 'email': 'user0002@example.com', 'username': 'Miles@example.onmicrosoft.com'}
Bauer Smith {'accountEnabled': False, 'email': 'user0003@example.com', 'username': 'Bauer@example.onmicrosoft.com'}
Adele Smith {'accountEnabled': True, 'email': 'user0004@example.com', 'username': 'Adele@example.onmicrosoft.com'}
Ruby Smith {'accountEnabled': True, 'email': None, 'username': 'rubysmith@example.onmicrosoft.com'}
```